

IT3071 – Machine Learning and Optimization Methods

Gradient Descent Algorithm

H.M. Samadhi Chathuranga Rathnayake

Gradient Decent

- Gradient descent (GD) is an iterative first-order optimisation algorithm, used to find a local minimum/maximum of a given function.
- This method is commonly used in machine learning (ML) and deep learning (DL) to minimise a cost/loss function.
- This method was proposed long before the era of modern computers by Augustin-Louis Cauchy in 1847.
- Since that time, there was an significant development in computer science and numerical methods.
- That led to numerous improved versions of Gradient Descent.

What is Gradient?

- Intuitively it is a slope of a curve at a given point in a specified direction.
- In the case of a univariate function, it is simply the first derivative at a selected point.
- In the case of a multivariate function, it is a vector of derivatives in each main direction (along variable axes).
- Because we are interested only in a slope along one axis and we don't care about others these derivatives are called partial derivatives.

What is Gradient?

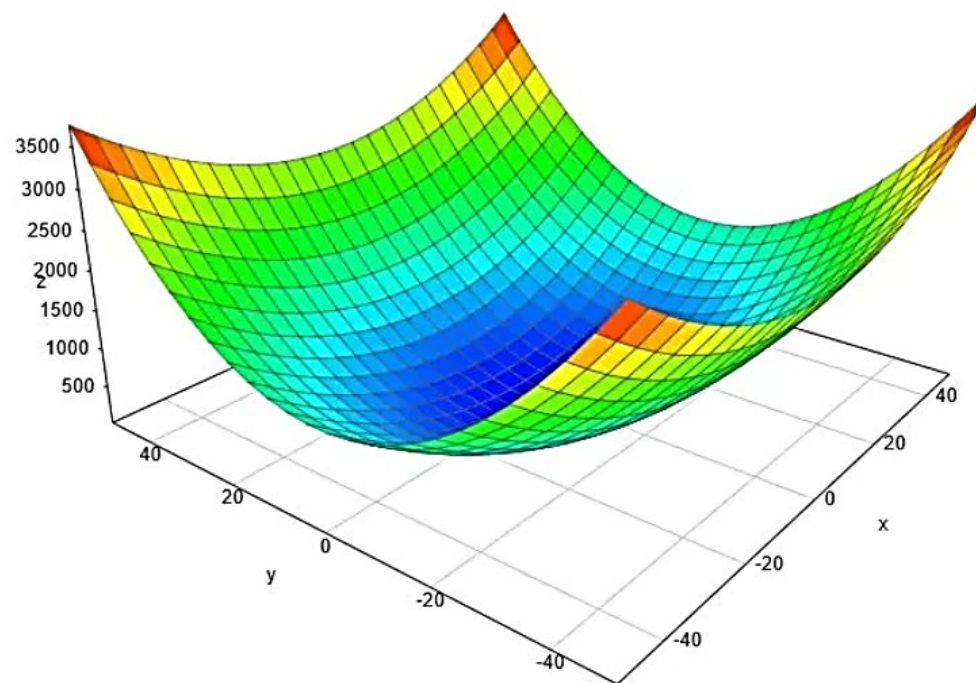
- A gradient for an n-dimensional function $f(x)$ at a given point p is defined as follows:

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

What is Gradient?

- Ex- Consider the following

$$f(x) = 0.5x^2 + y^2$$



What is Gradient?

- Ex- Let's assume we are interested in a gradient at point $p(10,10)$:

$$\frac{\partial f(x,y)}{\partial x} = x, \quad \frac{\partial f(x,y)}{\partial y} = 2y$$

$$\nabla f(x,y) = \begin{bmatrix} x \\ 2y \end{bmatrix}$$

$$\nabla f(10,10) = \begin{bmatrix} 10 \\ 20 \end{bmatrix}$$

Gradient Descent Algorithm (GD)

- Gradient Descent Algorithm iteratively calculates the next point using gradient at the current position, scales it (by a learning rate) and subtracts obtained value from the current position (makes a step).
- It subtracts the value because we want to minimize the function (to maximize it would be adding).
- This process can be written as:

$$p_{n+1} = p_n - \eta \nabla f(p_n)$$

Gradient Descent Algorithm

- η is the learning rate which scales the gradient and thus controls the step size.
- The smaller learning rate the longer GD converges, or may reach maximum iteration before reaching the optimum point
- If learning rate is too big the algorithm may not converge to the optimal point (jump around) or even to diverge completely.

Gradient Descent Algorithm

- Steps of GD
 1. Choose a starting point (initialization)
 2. Calculate gradient at this point
 3. Make a scaled step in the opposite direction to the gradient (objective: minimize)
 4. Repeat points 2 and 3 until one of the criteria is met:
 5. Maximum number of iterations reached
 6. Step size is smaller than the tolerance (due to scaling or a small gradient).

Gradient Descent Algorithm

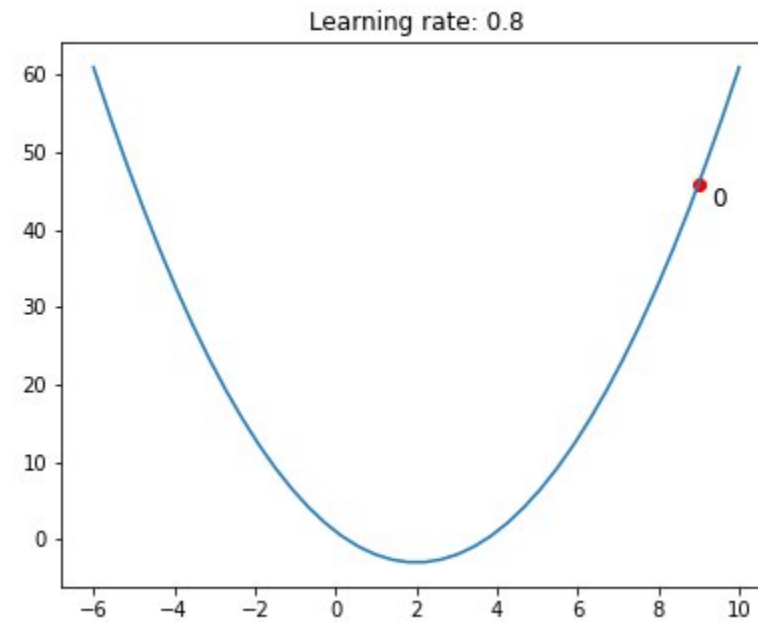
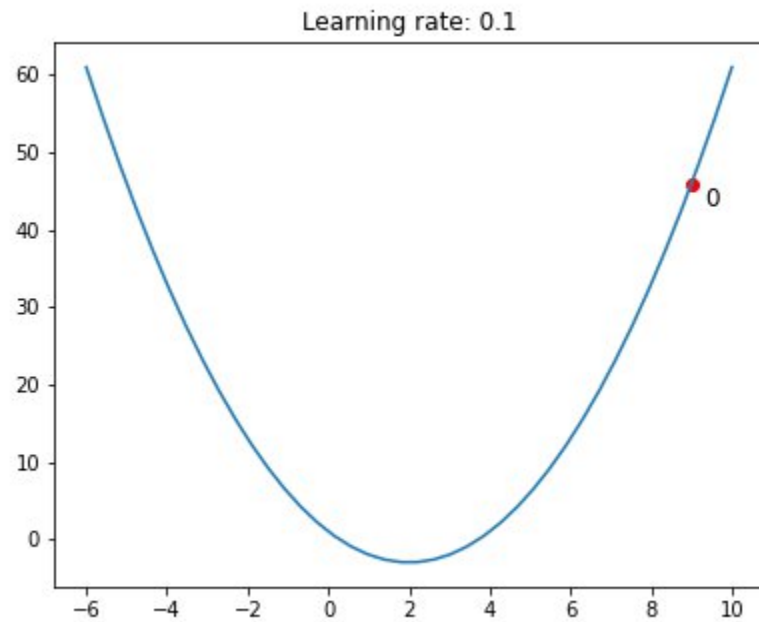
- Ex- $f(x) = x^2 - 4x + 1$

$$\frac{df(x)}{dx} = 2x - 4$$

$$\begin{aligned}x_1 &= 9 - 0.1 \cdot (2 \cdot 9 - 4) = 7.6 \\x_2 &= 7.6 - 0.1 \cdot (2 \cdot 7.6 - 4) = 6.48 \\x_3 &= 6.48 - 0.1 \cdot (2 \cdot 6.48 - 4) = 5.584\end{aligned}$$

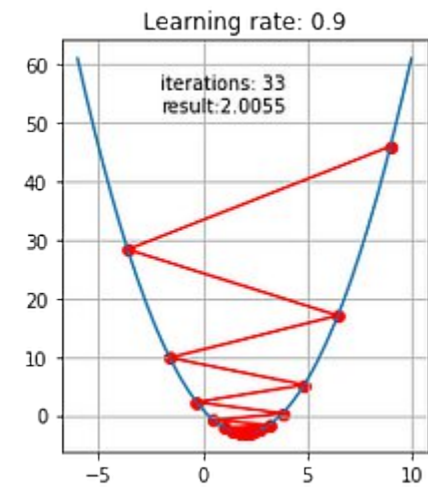
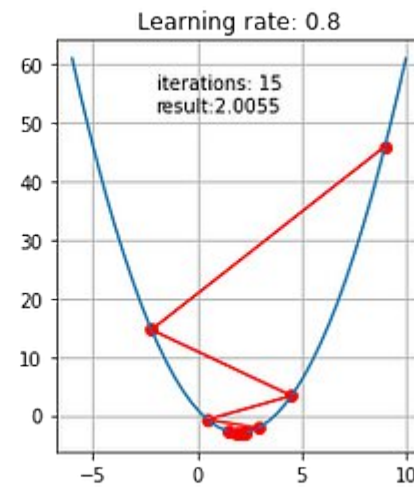
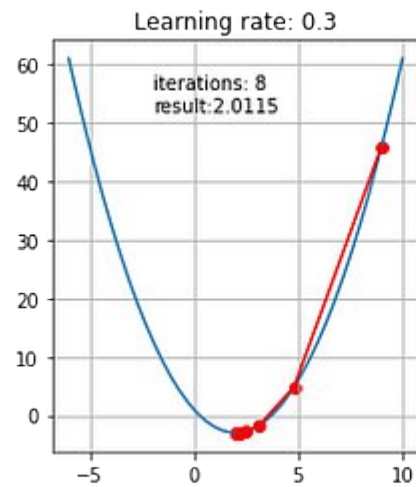
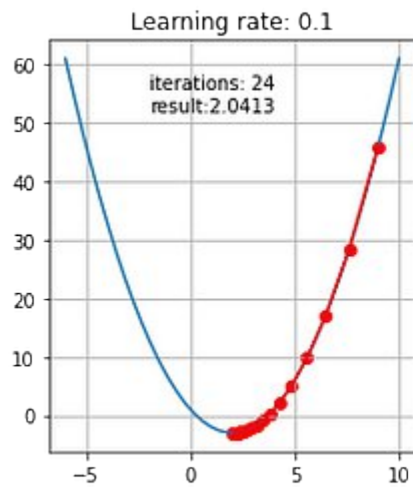
Gradient Descent Algorithm

- Ex-



Gradient Descent Algorithm

- Ex-



Minimizing Loss Function with GD

- The loss function

$$J(w, b) = \frac{1}{n}(y_p - y)^2$$

Gradient of $J(w, b)$ with respect to w

$$\begin{aligned} J'_w &= \frac{\partial J(w, b)}{\partial w} \\ &= \frac{\partial}{\partial w} \left[\frac{1}{n}(y_p - y)^2 \right] \\ &= \frac{2(y_p - y)}{n} \frac{\partial}{\partial w} [(y_p - y)] \\ &= \frac{2(y_p - y)}{n} \frac{\partial}{\partial w} [(xW^T + b) - y] \\ &= \frac{2(y_p - y)}{n} \left[\frac{\partial(xW^T + b)}{\partial w} - \frac{\partial(y)}{\partial w} \right] \\ &= \frac{2(y_p - y)}{n} [x - 0] \\ &= \frac{1}{n}(y_p - y)[2x] \end{aligned}$$

Gradient of $J(w, b)$ with respect to b

$$\begin{aligned} J'_b &= \frac{\partial J(w, b)}{\partial b} \\ &= \frac{\partial}{\partial b} \left[\frac{1}{n}(y_p - y)^2 \right] \\ &= \frac{2(y_p - y)}{n} \frac{\partial}{\partial b} [(y_p - y)] \\ &= \frac{2(y_p - y)}{n} \frac{\partial}{\partial b} [(xW^T + b) - y] \\ &= \frac{2(y_p - y)}{n} \left[\frac{\partial(xW^T + b)}{\partial b} - \frac{\partial(y)}{\partial b} \right] \\ &= \frac{2(y_p - y)}{n} [1 - 0] \\ &= \frac{1}{n}(y_p - y)[2] \end{aligned}$$

Minimizing Loss Function with GD

- The GD Algorithm is,

$$Param = Param - \gamma \nabla J$$

- So, the weights and bias values will be updated as,

$$\begin{aligned} w &= w - \gamma \nabla J(w, b) \\ b &= b - \gamma \nabla J(w, b) \end{aligned}$$

Gradient Descent Learning Rate

- The learning rate is a critical hyperparameter in the context of gradient descent, influencing the size of steps taken during the optimization process to update the model parameters.
- Choosing an appropriate learning rate is crucial for efficient and effective model training.
- When the learning rate is too small, the optimization process progresses very slowly.
- The model makes tiny updates to its parameters in each iteration, leading to sluggish convergence and potentially getting stuck in local minima.
- On the other hand, an excessively large learning rate can cause the optimization algorithm to overshoot the optimal parameter values, leading to divergence or oscillations that hinder convergence.
- Achieving the right balance is essential.

Advantages of GD

- **Widely used:** Gradient descent and its variants are widely used in machine learning and optimization problems because they are effective and easy to implement.
- **Convergence:** Gradient descent and its variants can converge to a global minimum or a good local minimum of the cost function, depending on the problem and the variant used.
- **Scalability:** Many variants of gradient descent can be parallelized and are scalable to large datasets and high-dimensional models.
- **Flexibility:** Different variants of gradient descent offer a range of trade-offs between accuracy and speed, and can be adjusted to optimize the performance of a specific problem.

Disadvantages of GD

- **Choice of learning rate:** The choice of learning rate is crucial for the convergence of gradient descent and its variants. Choosing a learning rate that is too large can lead to oscillations or overshooting while choosing a learning rate that is too small can lead to slow convergence or getting stuck in local minima.
- **Sensitivity to initialization:** Gradient descent and its variants can be sensitive to the initialization of the model's parameters, which can affect the convergence and the quality of the solution.
- **Time-consuming:** Gradient descent and its variants can be time-consuming, especially when dealing with large datasets and high-dimensional models. The convergence speed can also vary depending on the variant used and the specific problem.
- **Local optima:** Gradient descent and its variants can converge to a local minimum instead of the global minimum of the cost function, especially in non-convex problems. This can affect the quality of the solution, and techniques like random initialization and multiple restarts may be used to mitigate this issue.

Different Variants of Gradient Descent

- There are several variants of gradient descent that differ in the way the step size or learning rate is chosen and the way the updates are made. Here are some popular variants.
 - Batch Gradient Descent
 - Mini-batch Gradient Descent
 - Stochastic Gradient Descent (SGD)
 - Momentum-based Gradient Descent
 - Nesterov Accelerated Gradient (NAG)

Batch Gradient Descent

- In batch gradient descent, to update the model parameter values like weight and bias, the entire training dataset is used to compute the gradient and update the parameters at each iteration.
- This can be slow for large datasets but may lead to a more accurate model.
- It is effective for convex or relatively smooth error manifolds because it moves directly toward an optimal solution by taking a large step in the direction of the negative gradient of the cost function.
- However, it can be slow for large datasets because it computes the gradient and updates the parameters using the entire training dataset at each iteration.
- This can result in longer training times and higher computational costs.

Mini-batch Gradient Descent

- In Mini-batch gradient descent a small batch of training examples is used to compute the gradient and update the parameters at each iteration.
- This can be a good compromise between batch gradient descent and Stochastic Gradient Descent, as it can be faster than batch gradient descent and less noisy than Stochastic Gradient Descent.

Ex-

X_1	X_2	X_3	Y (Target)
1	2	3	10
2	1	0	8
3	2	1	12
0	1	2	6

- Consider the following regression model

$$\hat{Y} = w_1X_1 + w_2X_2 + w_3X_3 + b$$

- In the Batch GD, the model uses all 4 records to calculate the error and update the weights.
- Start with initial weights: $w_1=w_2=w_3=0$, $b=0$
- Consider the learning rate as 0.01

Ex-

- For each iteration:
 - Predict predictions for all 4 records.
 - Calculate the total loss (Ex- Mean Squared Error).
 - Compute gradient (how much to change weights) using all records.
 - Update the weights and bias once using those gradients.

Ex-

- Predictions for all 4 records

$$\hat{y}_i = 0 \quad \text{for all } i$$

- Loss (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$= \frac{1}{4} [(10 - 0)^2 + (8 - 0)^2 + (12 - 0)^2 + (6 - 0)^2] = \frac{1}{4} [100 + 64 + 144 + 36] = \frac{344}{4} = 86$$

- Compute Gradients

$$\frac{\partial \text{MSE}}{\partial w_j} = -\frac{2}{n} \sum_{i=1}^n x_{ij} (y_i - \hat{y}_i) \qquad \frac{\partial \text{MSE}}{\partial w_j} = -\frac{2}{4} \sum x_{ij} y_i$$

Ex-

- Compute gradients for each weight:

w_1 :

$$= -0.5[(1)(10) + (2)(8) + (3)(12) + (0)(6)] = -0.5[10 + 16 + 36 + 0] = -0.5 \times 62 = -31$$

w_2 :

$$= -0.5[(2)(10) + (1)(8) + (2)(12) + (1)(6)] = -0.5[20 + 8 + 24 + 6] = -0.5 \times 58 = -29$$

w_3 :

$$= -0.5[(3)(10) + (0)(8) + (1)(12) + (2)(6)] = -0.5[30 + 0 + 12 + 12] = -0.5 \times 54 = -27$$

- Bias:

$$\frac{\partial \text{MSE}}{\partial b} = -\frac{2}{n} \sum (y_i - \hat{y}_i) = -0.5(10 + 8 + 12 + 6) = -0.5 \times 36 = -18$$

Ex-

- Update Weights

$$w_j = w_j - \eta \cdot \frac{\partial \text{MSE}}{\partial w_j}$$

$$w_1 = 0 - 0.01 \cdot (-31) = 0.31 \quad w_2 = 0.29 \quad w_3 = 0.27 \quad b = 0.18$$

Ex-

- In Mini Batch GD, the model uses only 2 records at a time to learn.
- Mini Batch 1-

X_1	X_2	X_3	Y
1	2	3	10
2	1	0	8

- Predict Y for these 2 and compute gradients and update weights/bias.
- Mini Batch 2-

X_1	X_2	X_3	Y
3	2	1	12
0	1	2	6

- Do the same.

Stochastic Gradient Descent (SGD)

- Stochastic Gradient Descent (SGD) is a variant of the Gradient Descent algorithm that is used for optimizing machine learning models.
- It addresses the computational inefficiency of traditional Gradient Descent methods when dealing with large datasets in machine learning projects.
- In SGD, instead of using the entire dataset for each iteration, only a single random training example (or a small batch) is selected to calculate the gradient and update the model parameters.
- This random selection introduces randomness into the optimization process, hence the term “stochastic” in stochastic Gradient Descent.

Stochastic Gradient Descent (SGD)

- Stochastic Gradient Descent (SGD) is a version of gradient descent that updates the model using just one training example at a time,

$$w_{t+1} = w_t - \eta \cdot \nabla L(w_t; x^{(i)}, y^{(i)})$$

w_t = current weights

η = learning rate

$\nabla L(w_t; x^{(i)}, y^{(i)})$ = gradient of the loss using **only one** sample $(x^{(i)}, y^{(i)})$

Stochastic Gradient Descent (SGD)

- Ex-

X_1	X_2	X_3	Y
1	2	3	10
2	1	0	8
3	2	1	12
0	1	2	6

- In SGD,
- Pick one row, say $(2, 1, 0) \rightarrow Y=8$
- Compute the gradient from this one row
- Update weights immediately
- Move to another random row and repeat

Stochastic Gradient Descent (SGD)

- Simply
 - At each step, randomly pick one training sample.
 - Compute the gradient using just that one sample.
 - Immediately update the model parameters.
 - Repeat for every sample, then continue for more epochs.
- In SGD, since only one sample from the dataset is chosen at random for each iteration, the path taken by the algorithm to reach the minima is usually noisier than your typical Gradient Descent algorithm.
- But that doesn't matter all that much because the path taken by the algorithm does not matter, as long as we reach the minimum and with a significantly shorter training time.

Ex-

X_1	X_2	X_3	Y (Target)
1	2	3	10
2	1	0	8
3	2	1	12
0	1	2	6

- Consider the following regression model

$$\hat{Y} = w_1X_1 + w_2X_2 + w_3X_3 + b$$

- In the Batch GD, the model uses all 4 records to calculate the error and update the weights.
- Start with initial weights: $w_1=w_2=w_3=0$, $b=0$
- Consider the learning rate as 0.01

Ex-

- Record 1: $(1, 2, 3) \rightarrow Y = 10$

$$\hat{y} = 0 \quad \text{Error} = y - \hat{y} = 10 - 0 = 10$$

$$\frac{\partial L}{\partial w_1} = -2 \cdot 1 \cdot 10 = -20 \quad \frac{\partial L}{\partial w_2} = -2 \cdot 2 \cdot 10 = -40 \quad \frac{\partial L}{\partial w_3} = -2 \cdot 3 \cdot 10 = -60 \quad \frac{\partial L}{\partial b} = -2 \cdot 10 = -20$$

$$w_1 = 0 + 0.01 \cdot 20 = 0.20 \quad w_2 = 0.40 \quad w_3 = 0.60 \quad b = 0.20$$

Ex-

- Record 2: $(2, 1, 0) \rightarrow Y = 8$

$$\hat{y} = (0.20)(2) + (0.40)(1) + (0.60)(0) + 0.20 = 0.4 + 0.4 + 0 + 0.2 = 1.0 \quad \text{Error} = 8 - 1 = 7$$

$$\frac{\partial L}{\partial w_1} = -2 \cdot 2 \cdot 7 = -28 \quad \frac{\partial L}{\partial w_2} = -2 \cdot 1 \cdot 7 = -14 \quad \frac{\partial L}{\partial w_3} = 0 \quad \frac{\partial L}{\partial b} = -2 \cdot 7 = -14$$

$$w_1 = 0.20 + 0.01 \cdot 28 = 0.48 \quad w_2 = 0.40 + 0.14 = 0.54 \quad w_3 = 0.60(\text{unchanged}) \quad b = 0.20 + 0.14 = 0.34$$

- Continue the same process.

Comparing Main GD Methods

Method	Data Used per Update	Speed	Stability	Randomness	Good For
Batch GD	Whole dataset	Slow	Very stable	Low	Small datasets
Mini-batch GD	Small batches	Medium	Balanced	Medium	All datasets
Stochastic GD	One data point	Fast	Noisy	High	Large datasets, real-time training

SGD Algorithm

1. Initialization: Randomly initialize the parameters of the model.
2. Set Parameters: Determine the number of iterations and the learning rate (α) for updating the parameters.
3. Stochastic Gradient Descent Loop: Repeat the following steps until the model converges or reaches the maximum number of iterations:
 1. Shuffle the training dataset to introduce randomness.
 2. Iterate over each training example (or a small batch) in the shuffled order.
 3. Compute the gradient of the cost function with respect to the model parameters using the current training
 4. example (or batch).
 5. Update the model parameters by taking a step in the direction of the negative gradient, scaled by the learning rate.
 6. Evaluate the convergence criteria, such as the difference in the cost function between iterations of the gradient.
4. Return Optimized Parameters: Once the convergence criteria are met or the maximum number of iterations is reached, return the optimized model parameters.

Advantages of SGD

- **Speed:** SGD is faster than other variants of Gradient Descent such as Batch Gradient Descent and Mini-Batch Gradient Descent since it uses only one example to update the parameters.
- **Memory Efficiency:** Since SGD updates the parameters for each training example one at a time, it is memory-efficient and can handle large datasets that cannot fit into memory.
- **Avoidance of Local Minima:** Due to the noisy updates in SGD, it has the ability to escape from local minima and converges to a global minimum.

Disadvantages of SGD

- **Noisy updates:** The updates in SGD are noisy and have a high variance, which can make the optimization process less stable and lead to oscillations around the minimum.
- **Slow Convergence:** SGD may require more iterations to converge to the minimum since it updates the parameters for each training example one at a time.
- **Sensitivity to Learning Rate:** The choice of learning rate can be critical in SGD since using a high learning rate can cause the algorithm to overshoot the minimum, while a low learning rate can make the algorithm converge slowly.